Universidad Tecnológica de Tijuana

UNIVERSIDAD TECNOLÓGICA DE TIJUANA

INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE

DISEÑO Y CONEXIÓN DE LAS BASES DE DATOS EN EL PROYECTO:

NÚCLEO — GESTIÓN Y MONITOREO DEL AMBIENTE

PRESENTADO POR:

Arce Llamas Irvin de Jesus
González González Brandon
Salazar Urtuzuastegui Yadira Guadalupe

MATERIA:

Administración de Proyectos de TI

PROFESOR(A):

Laura Trejo Medina

GRUPO: 9C

Tijuana, Baja California, 24 de octubre del 2026

1. Descripción general del proyecto

Núcleo es un sistema portátil de monitoreo ambiental en tiempo real que sustituye la verificación manual y el registro en bitácora física por un monitoreo continuo y automatizado. Su objetivo es identificar en un máximo de **2 minutos** cualquier desviación de temperatura, humedad relativa o calidad del aire fuera de los rangos permitidos, para actuar antes de que derive en un incidente que comprometa la salud o la seguridad de las personas.

Núcleo se ofrece como un servicio para organizaciones que necesitan un control estricto de sus condiciones ambientales, por ejemplo una fábrica de moldeo, un laboratorio farmacéutico o una escuela/guardería; la prueba piloto se realizará en la UTT. El sistema se compone de los siguientes subsistemas:

- **Dispositivo IoT portátil.** Placa ESP32 DevKit con firmware en MicroPython, sensor DHT22 (temperatura y humedad relativa) y CCS811, cuya medición de CO2 equivalente (eCO2, en ppm) define la calidad del aire. Cuenta con pantalla OLED de 0.96" I2C, LED RGB de nivel (verde, amarillo, rojo), LED de conexión Wi-Fi, buzzer, batería de litio y carcasa impresa en 3D. Publica sus lecturas por MQTT con TLS (puerto 8883) al broker Eclipse Mosquitto.
- **Backend.** API REST en Django 5.2 LTS con Django REST Framework 3.18, con un canal en vivo (Django Channels y Redis) para las alertas. Usa **dos bases de datos**: TimescaleDB 2.31 (nucleo_iot) guarda las lecturas como serie de tiempo y PostgreSQL 17 (nucleo_db) guarda la información de negocio y los estados derivados de esas lecturas.
- **Aplicación web (React 19).** Dashboards en tiempo real, histórico y gráficas de tendencia, gestión de usuarios y dispositivos, umbrales, panel de alertas, reportes con IA y un agente de IA que responde en lenguaje natural.
- **Aplicación móvil (React Native 0.88 con Expo SDK 58).** Muestra al operador su dispositivo asignado y al supervisor los dispositivos de su área, con historial reciente y notificaciones push de alerta.

2. Requerimientos funcionales

Se retoman los requisitos funcionales del SRS que determinan qué datos se guardan y en qué base de datos.

I) Lectura automática de sensores ambientales

La unidad portátil toma lecturas de temperatura, humedad y calidad del aire de forma automática y continua, con un intervalo de muestreo configurable entre 5 y 120 segundos (30 por defecto). Durante los primeros 20 minutos después de cada arranque el CCS811 se encuentra en calentamiento: el dispositivo muestra “Calentando” y las lecturas de calidad del aire se guardan marcadas como no estabilizadas, sin generar alertas ni entrar en los reportes.

1. El sensor obtiene el valor ambiental.
2. El ESP32 recibe el dato, valida que esté dentro del rango físico del sensor y lo muestra en la pantalla OLED.
3. El ESP32 publica la lectura por MQTT hacia el broker Mosquitto.

Ejemplo: con un intervalo de 30 segundos, el DHT22 y el CCS811 se leen dos veces por minuto y cada valor se envía con su fecha y hora.

Referencia: SRS 3.2.1.

II) Alertas automáticas por niveles

Cada lectura se clasifica en Normal, Advertencia o Peligro según los límites del umbral. Al entrar en Advertencia o Peligro cambia el LED RGB (y en peligro suena el buzzer) y se notifica a los responsables en un máximo de 2 minutos. La alerta queda registrada con su nivel, estado (pendiente, atendida o descartada), quién la atendió y cuándo. Mientras exista una alerta pendiente del mismo umbral y nivel no se crea otra; si una advertencia escala a peligro se genera una nueva.

Ejemplo: si la temperatura rebasa el límite de peligro, el LED cambia a rojo y suena el buzzer; el backend crea la alerta y envía un correo a los supervisores del área, o a los administradores si el área no tiene supervisor.

Referencia: SRS 3.2.2.

III) Visualización multiplataforma

Los datos se consultan desde la web, la app móvil y la pantalla del dispositivo. Las lecturas se actualizan cada 30 segundos por consulta a la API REST y las alertas llegan al momento por el canal en vivo (WebSocket).

Ejemplo: el supervisor ve en el panel web y en la app el estado de los dispositivos de su área; el operador ve solo su dispositivo asignado.

Referencia: SRS 3.2.3.

IV) Control de usuarios y permisos

Cada usuario solo ve los datos de su propia organización. Los roles son:

- **Administrador de plataforma:** equipo de Núcleo; da de alta, baja o suspende organizaciones y crea a su primer administrador. No consulta lecturas ni modifica umbrales.
- **Administrador:** áreas, dispositivos, usuarios, permisos y umbrales de su organización.
- **Supervisor:** operadores de su área, intervalo de muestreo, umbrales y atención de alertas.
- **Analista:** histórico de mediciones y reportes con IA.
- **Operador:** monitoreo en tiempo real de su dispositivo asignado desde la app móvil.

Ejemplo: un operador no puede modificar umbrales ni ver dispositivos de otra área.

Referencia: SRS 3.2.4.

V) Validación de datos y umbrales

Para cada variable y dispositivo se define un par de límites de advertencia y uno de peligro, y el sistema valida que peligro mínimo ≤ advertencia mínimo < advertencia máximo ≤ peligro máximo. El backend acepta solo valores dentro del rango físico (−40 a 80 °C, 0 a 100 %HR y 0 a 10000 ppm de eCO2); una lectura inválida se descarta y las demás del mismo mensaje se guardan.

Ejemplo: advertencia fuera de 18 a 25 °C y peligro fuera de 15 a 28 °C en el laboratorio.

Referencia: SRS 3.2.5.

VI) Secuencia de operaciones

El procesamiento sigue un flujo ordenado que separa las lecturas de los registros de negocio.

Ejemplo: sensor → ESP32 → MQTT (TLS) → Mosquitto → backend Django (validación y comparación contra el umbral) → lectura en TimescaleDB → si está fuera de rango, alerta y notificaciones en PostgreSQL en una sola transacción → API REST → web y app móvil.

Referencia: SRS 3.2.6.

VII) Manejo de errores

Sin conexión, el dispositivo sigue leyendo los sensores, comparando contra los últimos umbrales recibidos y alertando localmente. La plataforma marca el dispositivo como “Sin conexión” cuando pasan más de 3 intervalos de muestreo sin recibir lecturas.

Ejemplo: si se cae el Wi-Fi, el dispositivo sigue sonando ante una temperatura alta aunque la alerta no llegue a la plataforma.

Referencia: SRS 3.2.7.

VIII) Almacenamiento en base de datos

Las lecturas se almacenan en TimescaleDB con su sensor, variable, valor, hora de medición, hora de recepción e indicador de estabilización; PostgreSQL conserva únicamente los estados y registros que se derivan de ellas. Las lecturas se guardan solo con conexión estable; no se almacenan de forma local para subirlas después.

Ejemplo: una lectura de 26.4 °C se guarda en nucleo_iot.lectura; si rebasa la advertencia, en nucleo_db se crea la alerta y se actualiza la ultima_conexion del dispositivo.

Referencia: SRS 3.2.10.

IX) Histórico y gráficas de tendencia

La plataforma consulta en TimescaleDB el histórico por dispositivo, variable y periodo, y lo muestra en gráficas de tendencia.

Ejemplo: consultar la humedad del área de producción de la última semana y ver en qué horarios se presentaron los picos.

Referencia: SRS 3.2.11.

X) Reportes con inteligencia artificial

Las cifras de los reportes (promedios, máximos, mínimos y alertas) se calculan en la base de datos y la IA solo redacta el texto; el PDF se genera en el backend. Los reportes se piden desde la plataforma, se solicitan al agente de IA o se programan con frecuencia diaria, semanal o mensual, área (o toda la organización) y destinatarios de la misma organización. Una tarea cron genera y envía los programados. Si la IA no está configurada, la API responde 503 y no se guarda el reporte.

Ejemplo: pedir al agente un reporte de la calidad del aire del mes de octubre en el área de moldeo.

Referencia: SRS 3.2.12.

3. Requerimientos no funcionales

Requisitos no funcionales del SRS con impacto directo en el diseño de las bases de datos:

SRS	Categoría	Requerimiento	Impacto en las bases de datos
3.3.1	Rendimiento	Notificar una desviación en un máximo de 2 minutos; agregar dispositivos y organizaciones sin modificar código.	Lecturas en hypertable de TimescaleDB con índice por sensor, variable y fecha; fecha_hora, recibida_en y fecha_envio para medir el tiempo.
3.3.2	Seguridad	Aislamiento por organización, base no expuesta a internet, contraseñas cifradas y usuarios de BD sin superusuario.	Todo cuelga de organizacion; nucleo_app con permisos solo sobre nucleo_db y un usuario propio sobre nucleo_iot.
3.3.3	Fiabilidad	La alerta y sus notificaciones se registran juntas; la base evita datos inválidos.	Transacción en PostgreSQL; la alerta conserva num, valor y fecha y hora de la lectura; llaves foráneas y CHECK de límites.
3.3.4	Disponibilidad	Disponibilidad del 99 % en el despliegue en Vultr.	nucleo_db y nucleo_iot en servicios separados: la carga de lecturas no afecta a la base de negocio.
3.3.5	Mantenibilidad	Registrar el estado de cada sensor y su última calibración.	estado_sensor y fecha_calibracion en sensor.

4. Arquitectura de datos

4.1 Dos bases de datos

Conforme al SRS, la información se divide en dos bases de datos que el backend administra con el ORM de Django:

Base	Motor	Qué guarda	Tablas
nucleo_db	PostgreSQL 17	Información de negocio: organizaciones, áreas, usuarios, teléfonos, dispositivos, sensores, umbrales, alertas, notificaciones, programaciones y reportes, más los estados derivados de las lecturas (estado y última conexión del dispositivo).	16
nucleo_iot	TimescaleDB 2.31 (servicio separado)	Lecturas de los dispositivos como serie de tiempo. De aquí salen el histórico, las gráficas de tendencia y las cifras de los reportes.	1 (hypertable)

Las lecturas son el dato que más crece: con un intervalo de 30 segundos y tres variables, cada dispositivo genera 8,640 registros al día. TimescaleDB particiona la tabla por tiempo (hypertable), lo que mantiene rápidas las consultas por periodo y deja a la base de negocio solo con registros de bajo volumen.

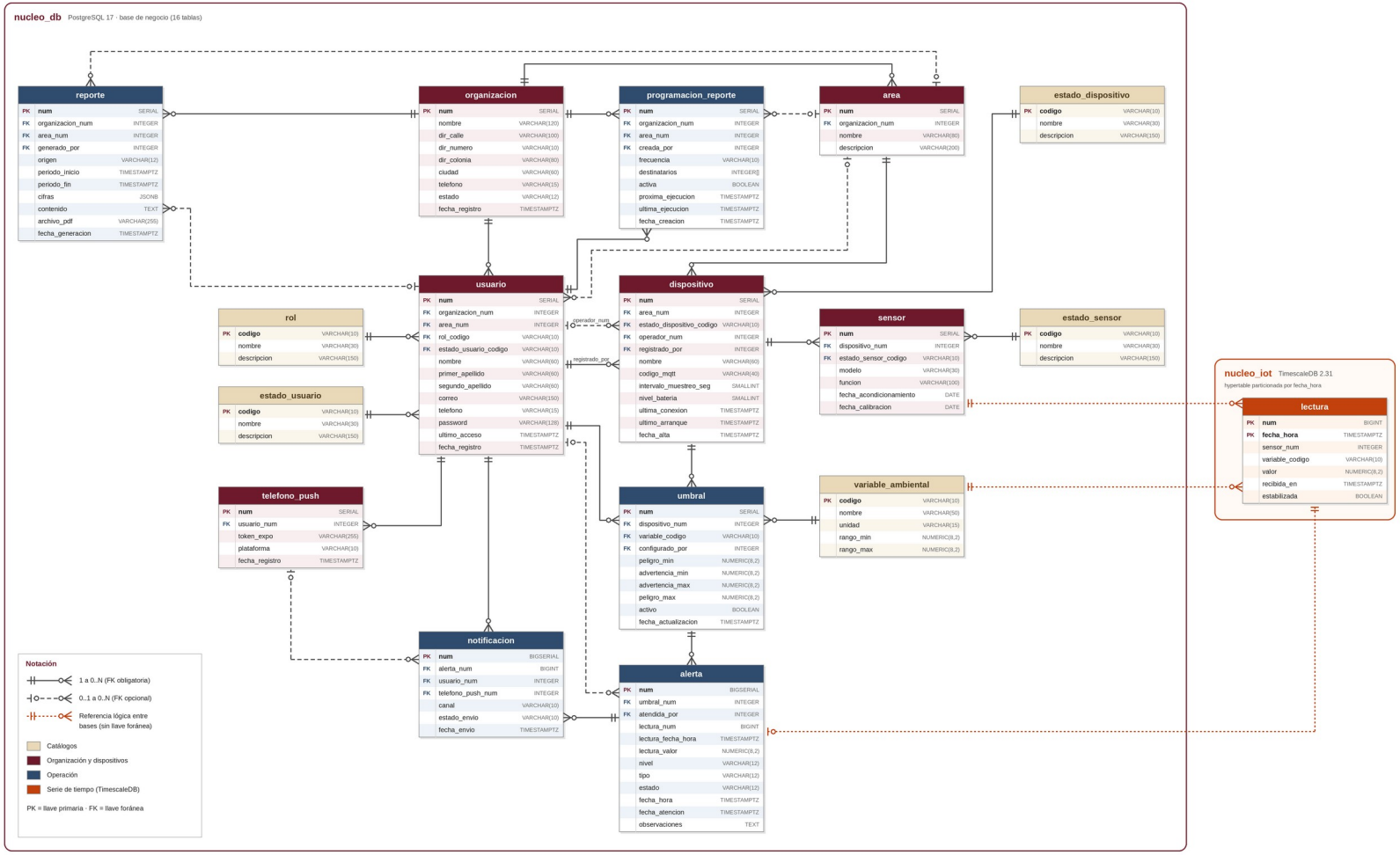
Como son dos bases distintas, **no existen llaves foráneas entre ellas**. Las columnas sensor_num y variable_codigo de la lectura son referencias lógicas que valida el backend antes de insertar. A su vez, la alerta copia el num, el valor y la fecha y hora de la lectura que la disparó, para que una alerta se pueda consultar completa sin depender de la otra base.

4.2 Cambio del modelo NoSQL al modelo relacional

El diseño anterior usaba colecciones NoSQL. Se cambió a un modelo relacional porque los datos están muy relacionados (una lectura pertenece a un sensor, que está en un dispositivo, en un área de una organización), porque las restricciones de la base evitan datos inválidos aunque falle una validación de la aplicación, porque la alerta y sus notificaciones deben registrarse en una sola transacción, y porque Django trabaja de forma nativa con PostgreSQL. TimescaleDB es una extensión de PostgreSQL, así que las lecturas también se consultan con SQL.

Colección NoSQL	Destino	Cambio principal
EdoUsuario / EdoDispositivo / EdoSensor	estado_usuario / estado_dispositivo / estado_sensor	Catálogos con llave primaria codigo; se agregan los estados Calentando y Sin conexión.
Rol	rol	Se agregan Administrador de plataforma y Analista.
TipoMedicion	variable_ambiental	Llave codigo (TEMP, HUM, AIRE) y rango físico aceptado.
Empresa	organizacion	El arreglo IDArea se sustituye por la llave foránea area.organizacion_num; se agrega su estado.
Area	area	Pertenece a una organización.
Empleado + Usuario	usuario	Una sola tabla, modelo de usuario de Django.
JornadaLaboral	(se elimina)	No corresponde a ningún requisito; el monitoreo es continuo.
Sensor	sensor	Se liga a su dispositivo; se agregan las fechas de acondicionamiento y calibración.
SensoresPorDispositivo	(se elimina)	Se reemplaza por la llave foránea sensor.dispositivo_num.
Umbral	umbral	Dos pares de límites (advertencia y peligro) por dispositivo y variable.
Medicion	lectura (nucleo_iot)	Se traslada a TimescaleDB como hypertable, con hora de recepción e indicador de estabilización.
Dispositivo	dispositivo	Se agregan codigo_mqtt, intervalo de muestreo, operador y último arranque.
(no existía)	telefono_push, alerta, notificacion, programacion_reporte, reporte	Nuevas tablas para alertas, notificaciones push y reportes con IA.

5. Modelo relacional



El marco granate es la base de negocio nucleo_db (PostgreSQL 17, 16 tablas) y el marco naranja es nucleo_iot (TimescaleDB 2.31). Las líneas naranjas punteadas son referencias lógicas entre bases: el backend las valida porque no puede haber llaves foráneas entre dos bases distintas.

6. Diccionario de datos

Por convención, las llaves primarias numéricas se llaman **num** y las de tipo texto **codigo**; las llaves foráneas llevan el nombre de la tabla a la que apuntan seguido de _num o _codigo (por ejemplo, area_num o rol_codigo). Las que indican una función conservan ese nombre (registrado_por, configurado_por, atendida_por, creada_por, generado_por).

6.A nucleo_db · PostgreSQL 17

6.1 rol

Catálogo de los roles del sistema: Administrador de plataforma, Administrador, Supervisor, Analista y Operador. Define qué puede ver y hacer cada usuario (3.2.4).

Campo	Tipo	Llave	Restricciones	Descripción
codigo	VARCHAR(10)	PK	NOT NULL	Código del rol (ADMINPLAT, ADMIN, SUPER, ANALISTA, OPER)
nombre	VARCHAR(30)		NOT NULL, UNIQUE	Nombre del rol
descripcion	VARCHAR(150)			Permisos que concede el rol

6.2 estado_usuario

Catálogo de estados de la cuenta. Solo inician sesión las cuentas activas (3.3.2).

Campo	Tipo	Llave	Restricciones	Descripción
codigo	VARCHAR(10)	PK	NOT NULL	Código del estado (ACT, INA, BLQ)
nombre	VARCHAR(30)		NOT NULL, UNIQUE	Nombre del estado
descripcion	VARCHAR(150)			Significado del estado

6.3 estado_dispositivo

Catálogo de estados del dispositivo, incluidos “Calentando” (primeros 20 minutos del CCS811) y “Sin conexión” (más de 3 intervalos sin lecturas).

Campo	Tipo	Llave	Restricciones	Descripción
codigo	VARCHAR(10)	PK	NOT NULL	Código del estado (LINEA, CALENT, SINCONEX, BATBAJA, MANT, BAJA)
nombre	VARCHAR(30)		NOT NULL, UNIQUE	Nombre del estado
descripcion	VARCHAR(150)			Significado del estado

6.4 estado_sensor

Catálogo de estados del sensor: Operando, Requiere calibración o Falla (3.3.5).

Campo	Tipo	Llave	Restricciones	Descripción
codigo	VARCHAR(10)	PK	NOT NULL	Código del estado (OPERA, CALIB, FALLA)
nombre	VARCHAR(30)		NOT NULL, UNIQUE	Nombre del estado
descripcion	VARCHAR(150)			Significado del estado

6.5 variable_ambiental

Catálogo de las variables medidas con su unidad y el rango físico que acepta el backend: temperatura de −40 a 80 °C, humedad de 0 a 100 %HR y calidad del aire (eCO2) de 0 a 10000 ppm (3.2.5).

Campo	Tipo	Llave	Restricciones	Descripción
codigo	VARCHAR(10)	PK	NOT NULL	Código de la variable (TEMP, HUM, AIRE)
nombre	VARCHAR(50)		NOT NULL	Nombre de la variable
unidad	VARCHAR(15)		NOT NULL	Unidad de medida (°C, %HR, ppm)
rango_min	NUMERIC(8,2)		NOT NULL	Valor físico mínimo aceptado
rango_max	NUMERIC(8,2)		NOT NULL	Valor físico máximo aceptado

Restricciones e índices: CHECK (rango_min < rango_max).

6.6 organizacion

Organización que contrata el servicio. El administrador de plataforma la da de alta, baja o suspende; solo inician sesión los usuarios de una organización activa.

Campo	Tipo	Llave	Restricciones	Descripción
num	SERIAL	PK	NOT NULL	Número identificador de la organización
nombre	VARCHAR(120)		NOT NULL	Nombre o razón social
dir_calle	VARCHAR(100)			Calle del domicilio
dir_numero	VARCHAR(10)			Número exterior
dir_colonia	VARCHAR(80)			Colonia
ciudad	VARCHAR(60)			Ciudad
telefono	VARCHAR(15)			Teléfono de contacto
estado	VARCHAR(12)		NOT NULL, DEFAULT 'ACTIVA'	ACTIVA, SUSPENDIDA o BAJA
fecha_registro	TIMESTAMPTZ		NOT NULL, DEFAULT now()	Fecha de alta

Restricciones e índices: CHECK (estado IN ('ACTIVA','SUSPENDIDA','BAJA')).

6.7 area

Áreas de una organización donde se instalan los dispositivos. El supervisor y el analista con área asignada solo ven los datos de su área.

Campo	Tipo	Llave	Restricciones	Descripción
num	SERIAL	PK	NOT NULL	Número identificador del área
organizacion_num	INTEGER	FK	NOT NULL → organizacion(num)	Organización a la que pertenece
nombre	VARCHAR(80)		NOT NULL	Nombre del área
descripcion	VARCHAR(200)			Ubicación o uso del área

Restricciones e índices: UNIQUE (organizacion_num, nombre).

6.8 usuario

Personas que acceden a la web o la app móvil; es el modelo de usuario de Django. El administrador de plataforma pertenece al equipo de Núcleo, por lo que es el único usuario sin organización.

Campo	Tipo	Llave	Restricciones	Descripción
num	SERIAL	PK	NOT NULL	Número identificador del usuario
organizacion_num	INTEGER	FK	NULL → organizacion(num)	Organización (NULL solo para ADMINPLAT)
area_num	INTEGER	FK	NULL → area(num)	Área asignada (opcional)
rol_codigo	VARCHAR(10)	FK	NOT NULL → rol(codigo)	Rol del usuario
estado_usuario_codigo	VARCHAR(10)	FK	NOT NULL → estado_usuario(codigo)	Estado de la cuenta
nombre	VARCHAR(60)		NOT NULL	Nombre(s)
primer_apellido	VARCHAR(60)		NOT NULL	Primer apellido
segundo_apellido	VARCHAR(60)			Segundo apellido
correo	VARCHAR(150)		NOT NULL, UNIQUE	Correo de acceso y de alertas
telefono	VARCHAR(15)			Teléfono celular
password	VARCHAR(128)		NOT NULL	Contraseña cifrada (hash de Django)
ultimo_acceso	TIMESTAMPTZ			Último inicio de sesión
fecha_registro	TIMESTAMPTZ		NOT NULL, DEFAULT now()	Fecha de alta

Restricciones e índices: CHECK ((rol_codigo = 'ADMINPLAT') = (organizacion_num IS NULL)).

6.9 telefono_push

Teléfonos registrados para notificaciones push con Expo. El token se registra al iniciar sesión en la app y se elimina al cerrarla; un usuario puede tener varios teléfonos.

Campo	Tipo	Llave	Restricciones	Descripción
num	SERIAL	PK	NOT NULL	Número identificador del registro
usuario_num	INTEGER	FK	NOT NULL → usuario(num)	Dueño del teléfono
token_expo	VARCHAR(255)		NOT NULL, UNIQUE	Token de Expo Push Notifications
plataforma	VARCHAR(10)		NOT NULL	ANDROID o IOS
fecha_registro	TIMESTAMPTZ		NOT NULL, DEFAULT now()	Inicio de sesión que lo registró

Restricciones e índices: CHECK (plataforma IN ('ANDROID','IOS')); ON DELETE CASCADE en usuario_num.

6.10 dispositivo

Unidades portátiles ESP32. Guarda su codigo_mqtt (client_id y tópico), el intervalo de muestreo, el operador asignado y los estados derivados de las lecturas: última conexión, batería y último arranque.

Campo	Tipo	Llave	Restricciones	Descripción
num	SERIAL	PK	NOT NULL	Número identificador del dispositivo
area_num	INTEGER	FK	NOT NULL → area(num)	Área donde está instalado
estado_dispositivo_codigo	VARCHAR(10)	FK	NOT NULL → estado_dispositivo(codigo)	Estado actual
operador_num	INTEGER	FK	NULL → usuario(num)	Operador asignado (app móvil)
registrado_por	INTEGER	FK	NOT NULL → usuario(num)	Usuario que lo dio de alta
nombre	VARCHAR(60)		NOT NULL	Nombre descriptivo
codigo_mqtt	VARCHAR(40)		NOT NULL, UNIQUE	client_id y tópico nucleo/devices/<codigo_mqtt>
intervalo_muestreo_seg	SMALLINT		NOT NULL, DEFAULT 30	Segundos entre lecturas
nivel_bateria	SMALLINT			Porcentaje de batería reportado
ultima_conexion	TIMESTAMPTZ			Última vez que envió lecturas
ultimo_arranque	TIMESTAMPTZ			Inicio de los 20 min de calentamiento
fecha_alta	TIMESTAMPTZ		NOT NULL, DEFAULT now()	Fecha de registro

Restricciones e índices: CHECK (intervalo_muestreo_seg BETWEEN 5 AND 120); CHECK (nivel_bateria BETWEEN 0 AND 100).

6.11 sensor

Sensores montados en cada dispositivo (DHT22 y CCS811), con su estado y fechas de acondicionamiento (48 horas antes de la primera puesta en servicio del CCS811) y de última calibración.

Campo	Tipo	Llave	Restricciones	Descripción
num	SERIAL	PK	NOT NULL	Número identificador del sensor
dispositivo_num	INTEGER	FK	NOT NULL → dispositivo(num)	Dispositivo en el que está montado
estado_sensor_codigo	VARCHAR(10)	FK	NOT NULL → estado_sensor(codigo)	Estado del sensor
modelo	VARCHAR(30)		NOT NULL	DHT22 o CCS811
funcion	VARCHAR(100)			Qué mide el sensor
fecha_acondicionamiento	DATE			Fin del acondicionamiento de 48 h
fecha_calibracion	DATE			Última calibración

Restricciones e índices: UNIQUE (dispositivo_num, modelo).

6.12 umbral

Límites de advertencia y de peligro de cada variable por dispositivo (3.2.5). La base de datos valida que peligro_min ≤ advertencia_min < advertencia_max ≤ peligro_max.

Campo	Tipo	Llave	Restricciones	Descripción
num	SERIAL	PK	NOT NULL	Número identificador del umbral
dispositivo_num	INTEGER	FK	NOT NULL → dispositivo(num)	Dispositivo al que aplica
variable_codigo	VARCHAR(10)	FK	NOT NULL → variable_ambiental(codigo)	Variable que limita
configurado_por	INTEGER	FK	NOT NULL → usuario(num)	Usuario que lo definió
peligro_min	NUMERIC(8,2)		NOT NULL	Límite inferior de peligro
advertencia_min	NUMERIC(8,2)		NOT NULL	Límite inferior de advertencia
advertencia_max	NUMERIC(8,2)		NOT NULL	Límite superior de advertencia
peligro_max	NUMERIC(8,2)		NOT NULL	Límite superior de peligro
activo	BOOLEAN		NOT NULL, DEFAULT TRUE	Si se evalúa o no
fecha_actualizacion	TIMESTAMPTZ		NOT NULL, DEFAULT now()	Última modificación

Restricciones e índices: UNIQUE (dispositivo_num, variable_codigo); CHECK (peligro_min <= advertencia_min AND advertencia_min < advertencia_max AND advertencia_max <= peligro_max).

6.13 alerta

Se crea cuando una lectura entra en Advertencia o Peligro (3.2.2). Como la lectura vive en TimescaleDB, la alerta conserva su identificador, valor y fecha y hora. Un índice único parcial impide otra alerta pendiente del mismo umbral y nivel.

Campo	Tipo	Llave	Restricciones	Descripción
num	BIGSERIAL	PK	NOT NULL	Número identificador de la alerta
umbral_num	INTEGER	FK	NOT NULL → umbral(num)	Umbral que se rebasó
atendida_por	INTEGER	FK	NULL → usuario(num)	Responsable que la atendió
lectura_num	BIGINT		NOT NULL	num de la lectura en nucleo_iot
lectura_fecha_hora	TIMESTAMPTZ		NOT NULL	fecha_hora de la lectura
lectura_valor	NUMERIC(8,2)		NOT NULL	Valor que disparó la alerta
nivel	VARCHAR(12)		NOT NULL	ADVERTENCIA o PELIGRO
tipo	VARCHAR(12)		NOT NULL	POR_ENCIMA o POR_DEBAJO
estado	VARCHAR(12)		NOT NULL, DEFAULT 'PENDIENTE'	PENDIENTE, ATENDIDA o DESCARTADA
fecha_hora	TIMESTAMPTZ		NOT NULL, DEFAULT now()	Cuándo se generó
fecha_atencion	TIMESTAMPTZ			Cuándo se atendió
observaciones	TEXT			Acciones tomadas

Restricciones e índices: CHECK (nivel IN ('ADVERTENCIA','PELIGRO')); CHECK (tipo IN ('POR_ENCIMA','POR_DEBAJO')); CHECK (estado IN ('PENDIENTE','ATENDIDA','DESCARTADA')); UNIQUE parcial (umbral_num, nivel) WHERE estado = 'PENDIENTE'.

6.14 notificacion

Cada aviso enviado por una alerta: correo a los supervisores del área (o a los administradores si no hay supervisor) y push a cada teléfono registrado. Su hora de envío comprueba los 2 minutos (3.3.1).

Campo	Tipo	Llave	Restricciones	Descripción
num	BIGSERIAL	PK	NOT NULL	Número identificador de la notificación
alerta_num	BIGINT	FK	NOT NULL → alerta(num)	Alerta que la originó
usuario_num	INTEGER	FK	NOT NULL → usuario(num)	Destinatario
telefono_push_num	INTEGER	FK	NULL → telefono_push(num)	Teléfono destino (solo PUSH)
canal	VARCHAR(10)		NOT NULL	CORREO o PUSH
estado_envio	VARCHAR(10)		NOT NULL	ENVIADA o FALLIDA
fecha_envio	TIMESTAMPTZ		NOT NULL, DEFAULT now()	Cuándo se envió

Restricciones e índices: CHECK (canal IN ('CORREO','PUSH')); CHECK (estado_envio IN ('ENVIADA','FALLIDA')); CHECK (canal = 'PUSH' OR telefono_push_num IS NULL); ON DELETE SET NULL en telefono_push_num.

6.15 programacion_reporte

Reportes periódicos programados por un administrador, supervisor o analista: frecuencia, área (o toda la organización) y destinatarios. Una tarea cron revisa las programaciones activas y genera los reportes (3.2.12).

Campo	Tipo	Llave	Restricciones	Descripción
`num`	SERIAL	PK	NOT NULL	Número identificador de la programación
`organizacion_num`	INTEGER	FK	NOT NULL → organizacion(num)	Organización
`area_num`	INTEGER	FK	NULL → area(num)	Área (NULL = toda la organización)
`creada_por`	INTEGER	FK	NOT NULL → usuario(num)	Usuario que la programó
`frecuencia`	VARCHAR(10)		NOT NULL	DIARIA, SEMANAL o MENSUAL
`destinatarios`	INTEGER[]		NOT NULL	num de los usuarios que reciben el correo
`activa`	BOOLEAN		NOT NULL, DEFAULT TRUE	Si el cron la ejecuta
`proxima_ejecucion`	TIMESTAMPTZ		NOT NULL	Siguiente generación
`ultima_ejecucion`	TIMESTAMPTZ			Última generación
`fecha_creacion`	TIMESTAMPTZ		NOT NULL, DEFAULT now()	Fecha de alta

Restricciones e índices: CHECK (frecuencia IN ('DIARIA','SEMANAL','MENSUAL')); CHECK (cardinality(destinatarios) > 0).

6.16 reporte

Reportes con inteligencia artificial. Las cifras se calculan en la base de datos (consultas a nucleo_iot) y se guardan en JSONB; la IA solo redacta el texto y el PDF se genera en el backend. Si la IA no está configurada, no se guarda el reporte.

Campo	Tipo	Llave	Restricciones	Descripción
num	SERIAL	PK	NOT NULL	Número identificador del reporte
organizacion_num	INTEGER	FK	NOT NULL → organizacion(num)	Organización del reporte
area_num	INTEGER	FK	NULL → area(num)	Área (NULL = toda la organización)
generado_por	INTEGER	FK	NULL → usuario(num)	Quién lo pidió (NULL si fue programado)
origen	VARCHAR(12)		NOT NULL	MANUAL, PROGRAMADO o AGENTE
periodo_inicio	TIMESTAMPTZ		NOT NULL	Inicio del periodo analizado
periodo_fin	TIMESTAMPTZ		NOT NULL	Fin del periodo analizado
cifras	JSONB		NOT NULL	Promedios, máximos, mínimos y alertas
contenido	TEXT		NOT NULL	Texto redactado por la IA
archivo_pdf	VARCHAR(255)			Ruta del PDF generado
fecha_generacion	TIMESTAMPTZ		NOT NULL, DEFAULT now()	Cuándo se generó

Restricciones e índices: CHECK (origen IN ('MANUAL','PROGRAMADO','AGENTE')); CHECK (periodo_inicio < periodo_fin).

6.B nucleo_iot · TimescaleDB 2.31

6.17 lectura

Cada medición recibida por MQTT, guardada como serie de tiempo en TimescaleDB (hypertable particionada por fecha_hora). sensor_num y variable_codigo hacen referencia lógica a nucleo_db: no hay llave foránea entre bases, el backend las valida.

Campo	Tipo	Llave	Restricciones	Descripción
num	BIGINT	PK	NOT NULL, IDENTITY	Número identificador de la lectura
fecha_hora	TIMESTAMPTZ	PK	NOT NULL	Momento de la medición (partición)
sensor_num	INTEGER		NOT NULL	Ref. lógica a nucleo_db.sensor(num)
variable_codigo	VARCHAR(10)		NOT NULL	Ref. lógica a variable_ambiental(codigo)
valor	NUMERIC(8,2)		NOT NULL	Valor dentro del rango físico
recibida_en	TIMESTAMPTZ		NOT NULL, DEFAULT now()	Llegada al servidor
estabilizada	BOOLEAN		NOT NULL, DEFAULT TRUE	FALSE en el calentamiento del CCS811

Restricciones e índices: PRIMARY KEY (num, fecha_hora); CHECK (variable_codigo IN ('TEMP','HUM','AIRE')); CHECK de rango físico por variable; INDEX (sensor_num, variable_codigo, fecha_hora DESC).

7. Conexión de las bases de datos

7.1 Configuración

En producción, nucleo_db se aloja en el mismo servidor virtual Linux de Vultr que la API y el broker; escucha solo en localhost (puerto 5432, UTF-8) y no queda expuesta a internet. nucleo_iot corre en un servicio separado de TimescaleDB. Las aplicaciones web y móvil nunca se conectan a las bases ni al broker: todo pasa por la API REST con HTTPS. Para desarrollo y demostraciones se mantiene una instalación en localhost con la misma configuración. Las credenciales se leen de variables de entorno (archivo .env que no se sube a GitHub):

```
# settings.py: conexión de Django con las dos bases de datos
import os

DATABASES = {
    "default": {                        # nucleo_db (PostgreSQL 17)
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "nucleo_db",
        "USER": os.getenv("DB_USER", "nucleo_app"),
        "PASSWORD": os.getenv("DB_PASSWORD"),
        "HOST": os.getenv("DB_HOST", "localhost"),
        "PORT": os.getenv("DB_PORT", "5432"),
        "CONN_MAX_AGE": 60,
    },
    "iot": {                            # nucleo_iot (TimescaleDB 2.31)
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "nucleo_iot",
        "USER": os.getenv("IOT_DB_USER"),
        "PASSWORD": os.getenv("IOT_DB_PASSWORD"),
        "HOST": os.getenv("IOT_DB_HOST", "localhost"),
        "PORT": os.getenv("IOT_DB_PORT", "5433"),
    },
}
DATABASE_ROUTERS = ["core.routers.LecturaRouter"]

AUTH_USER_MODEL = "usuarios.Usuario"    # tabla usuario
TIME_ZONE = "America/Tijuana"
USE_TZ = True                           # fechas guardadas en UTC
```

Un router de Django indica qué modelos viven en cada base: el modelo Lectura usa la conexión iot y el resto usa default. Así cada base recibe solo sus propias migraciones.

```
# core/routers.py: envía el modelo Lectura a nucleo_iot
class LecturaRouter:
    app_iot = "lecturas"

    def db_for_read(self, model, **hints):
        return "iot" if model._meta.app_label == self.app_iot else "default"

    db_for_write = db_for_read

    def allow_relation(self, obj1, obj2, **hints):
        # no hay relaciones entre modelos de bases distintas
        return obj1._state.db == obj2._state.db

    def allow_migrate(self, db, app_label, **hints):
        return (db == "iot") == (app_label == self.app_iot)
```

El backend usa usuarios de base de datos sin privilegios de superusuario: nucleo_app con permisos solo sobre nucleo_db y un usuario propio con permisos solo sobre nucleo_iot. La hypertable se crea con una migración que ejecuta create_hypertable después de crear la tabla.

7.2 Flujo de una lectura

1. El ESP32 publica un mensaje JSON en el tópico nucleo/devices/<codigo_mqtt>/measurements, con TLS en el puerto 8883. Mosquitto no permite acceso anónimo y cada dispositivo solo publica en su propio tópico.
2. El backend recibe el mensaje, valida cada valor contra el rango físico de su variable y descarta solo las lecturas inválidas. Si no viene la fecha y hora, usa la hora de recepción.
3. Las lecturas válidas se insertan en **nucleo_iot.lectura**; las de calidad del aire durante el calentamiento se guardan con estabilizada = FALSE.
4. En **nucleo_db** se actualizan ultima_conexion, nivel_bateria y el estado del dispositivo.
5. Cada lectura estabilizada se compara con el umbral activo de su dispositivo y variable. Si entra en Advertencia o Peligro y no hay una alerta pendiente del mismo umbral y nivel, se registran la alerta y sus notificaciones en una sola transacción de nucleo_db.

6. Se envían el correo (y el push cuando se active), y la alerta se publica por el canal en vivo (Django Channels con Redis) a la web y la app.
7. El backend publica el intervalo de muestreo y los umbrales como mensaje retenido en nucleo/devices/<codigo_mqtt>/config, para que el dispositivo los use también sin conexión.

Ejemplo del mensaje que publica el dispositivo:

```
Tópico: nucleo/devices/nuc001/measurements
{
  "dispositivo": "nuc001",
  "fecha_hora": "2026-10-24T08:30:00-07:00",
  "bateria": 87,
  "aire_estabilizado": true,
  "lecturas": [
    {"sensor": "DHT22",  "variable": "TEMP", "valor": 26.4},
    {"sensor": "DHT22",  "variable": "HUM",  "valor": 48.0},
    {"sensor": "CCS811", "variable": "AIRE", "valor": 612}
  ]
}
```

Anexo A. Script de nucleo_db (PostgreSQL 17)

Crea las 16 tablas de la base de negocio con sus llaves, restricciones e índices, e inserta los códigos iniciales de los catálogos. Se probó en PostgreSQL de forma local, incluidas las restricciones de umbrales, del administrador de plataforma y de alertas pendientes duplicadas.

```
-- ============================================================
-- Núcleo · nucleo_db (PostgreSQL 17) · base de negocio
-- 16 tablas: catálogos, organización, dispositivos y operación
-- ============================================================

-- ---------- Catálogos ----------
CREATE TABLE rol (
    codigo      VARCHAR(10)  PRIMARY KEY,
    nombre      VARCHAR(30)  NOT NULL UNIQUE,
    descripcion VARCHAR(150)
);

CREATE TABLE estado_usuario (
    codigo      VARCHAR(10)  PRIMARY KEY,
    nombre      VARCHAR(30)  NOT NULL UNIQUE,
    descripcion VARCHAR(150)
);

CREATE TABLE estado_dispositivo (
    codigo      VARCHAR(10)  PRIMARY KEY,
    nombre      VARCHAR(30)  NOT NULL UNIQUE,
    descripcion VARCHAR(150)
);

CREATE TABLE estado_sensor (
    codigo      VARCHAR(10)  PRIMARY KEY,
    nombre      VARCHAR(30)  NOT NULL UNIQUE,
    descripcion VARCHAR(150)
);

CREATE TABLE variable_ambiental (
    codigo    VARCHAR(10)  PRIMARY KEY,
    nombre    VARCHAR(50)  NOT NULL,
    unidad    VARCHAR(15)  NOT NULL,
    rango_min NUMERIC(8,2) NOT NULL,
    rango_max NUMERIC(8,2) NOT NULL,
    CHECK (rango_min < rango_max)
);

-- ---------- Organización y dispositivos ----------
CREATE TABLE organizacion (
    num            SERIAL       PRIMARY KEY,
    nombre         VARCHAR(120) NOT NULL,
    dir_calle      VARCHAR(100),
```

```
    dir_numero     VARCHAR(10),
    dir_colonia    VARCHAR(80),
    ciudad         VARCHAR(60),
    telefono       VARCHAR(15),
    estado         VARCHAR(12)  NOT NULL DEFAULT 'ACTIVA',
    fecha_registro TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CHECK (estado IN ('ACTIVA','SUSPENDIDA','BAJA'))
);

CREATE TABLE area (
    num              SERIAL       PRIMARY KEY,
    organizacion_num INTEGER      NOT NULL REFERENCES organizacion(num),
    nombre           VARCHAR(80)  NOT NULL,
    descripcion      VARCHAR(200),
    UNIQUE (organizacion_num, nombre)
);

CREATE TABLE usuario (
    num                   SERIAL       PRIMARY KEY,
    organizacion_num      INTEGER      REFERENCES organizacion(num),
    area_num              INTEGER      REFERENCES area(num),
    rol_codigo            VARCHAR(10)  NOT NULL REFERENCES rol(codigo),
    estado_usuario_codigo VARCHAR(10)  NOT NULL REFERENCES estado_usuario(codigo),
    nombre                VARCHAR(60)  NOT NULL,
    primer_apellido       VARCHAR(60)  NOT NULL,
    segundo_apellido      VARCHAR(60),
    correo                VARCHAR(150) NOT NULL UNIQUE,
    telefono              VARCHAR(15),
    password              VARCHAR(128) NOT NULL,
    ultimo_acceso         TIMESTAMPTZ,
    fecha_registro        TIMESTAMPTZ  NOT NULL DEFAULT now(),
    -- solo el administrador de plataforma no pertenece a una organización
    CHECK ((rol_codigo = 'ADMINPLAT') = (organizacion_num IS NULL))
);

CREATE TABLE telefono_push (
    num            SERIAL       PRIMARY KEY,
    usuario_num    INTEGER      NOT NULL REFERENCES usuario(num) ON DELETE CASCADE,
    token_expo     VARCHAR(255) NOT NULL UNIQUE,
    plataforma     VARCHAR(10)  NOT NULL,
    fecha_registro TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CHECK (plataforma IN ('ANDROID','IOS'))
);

CREATE TABLE dispositivo (
    num                       SERIAL      PRIMARY KEY,
    area_num                  INTEGER     NOT NULL REFERENCES area(num),
    estado_dispositivo_codigo VARCHAR(10) NOT NULL REFERENCES estado_dispositivo(codigo),
    operador_num              INTEGER     REFERENCES usuario(num),
    registrado_por            INTEGER     NOT NULL REFERENCES usuario(num),
    nombre                    VARCHAR(60) NOT NULL,
    codigo_mqtt               VARCHAR(40) NOT NULL UNIQUE,
    intervalo_muestreo_seg    SMALLINT    NOT NULL DEFAULT 30,
    nivel_bateria             SMALLINT,
```

```
    ultima_conexion           TIMESTAMPTZ,
    ultimo_arranque           TIMESTAMPTZ,
    fecha_alta                TIMESTAMPTZ NOT NULL DEFAULT now(),
    CHECK (intervalo_muestreo_seg BETWEEN 5 AND 120),
    CHECK (nivel_bateria BETWEEN 0 AND 100)
);

CREATE TABLE sensor (
    num                     SERIAL       PRIMARY KEY,
    dispositivo_num         INTEGER      NOT NULL REFERENCES dispositivo(num),
    estado_sensor_codigo    VARCHAR(10)  NOT NULL REFERENCES estado_sensor(codigo),
    modelo                  VARCHAR(30)  NOT NULL,
    funcion                 VARCHAR(100),
    fecha_acondicionamiento DATE,
    fecha_calibracion       DATE,
    UNIQUE (dispositivo_num, modelo)
);

-- ---------- Operación ----------
CREATE TABLE umbral (
    num                 SERIAL       PRIMARY KEY,
    dispositivo_num     INTEGER      NOT NULL REFERENCES dispositivo(num),
    variable_codigo     VARCHAR(10)  NOT NULL REFERENCES variable_ambiental(codigo),
    configurado_por     INTEGER      NOT NULL REFERENCES usuario(num),
    peligro_min         NUMERIC(8,2) NOT NULL,
    advertencia_min     NUMERIC(8,2) NOT NULL,
    advertencia_max     NUMERIC(8,2) NOT NULL,
    peligro_max         NUMERIC(8,2) NOT NULL,
    activo              BOOLEAN      NOT NULL DEFAULT TRUE,
    fecha_actualizacion TIMESTAMPTZ  NOT NULL DEFAULT now(),
    UNIQUE (dispositivo_num, variable_codigo),
    CHECK (peligro_min <= advertencia_min
           AND advertencia_min < advertencia_max
           AND advertencia_max <= peligro_max)
);

CREATE TABLE alerta (
    num                BIGSERIAL    PRIMARY KEY,
    umbral_num         INTEGER      NOT NULL REFERENCES umbral(num),
    atendida_por       INTEGER      REFERENCES usuario(num),
    lectura_num        BIGINT       NOT NULL,  -- lectura.num en nucleo_iot
    lectura_fecha_hora TIMESTAMPTZ  NOT NULL,  -- lectura.fecha_hora en nucleo_iot
    lectura_valor      NUMERIC(8,2) NOT NULL,
    nivel              VARCHAR(12)  NOT NULL,
    tipo               VARCHAR(12)  NOT NULL,
    estado             VARCHAR(12)  NOT NULL DEFAULT 'PENDIENTE',
    fecha_hora         TIMESTAMPTZ  NOT NULL DEFAULT now(),
    fecha_atencion     TIMESTAMPTZ,
    observaciones      TEXT,
    CHECK (nivel IN ('ADVERTENCIA','PELIGRO')),
    CHECK (tipo IN ('POR_ENCIMA','POR_DEBAJO')),
    CHECK (estado IN ('PENDIENTE','ATENDIDA','DESCARTADA'))
);
```

```
-- no se repite una alerta pendiente del mismo umbral y nivel
CREATE UNIQUE INDEX uq_alerta_pendiente
    ON alerta (umbral_num, nivel) WHERE estado = 'PENDIENTE';
CREATE INDEX idx_alerta_estado ON alerta (estado, fecha_hora DESC);

CREATE TABLE notificacion (
    num               BIGSERIAL   PRIMARY KEY,
    alerta_num        BIGINT      NOT NULL REFERENCES alerta(num),
    usuario_num       INTEGER     NOT NULL REFERENCES usuario(num),
    telefono_push_num INTEGER     REFERENCES telefono_push(num) ON DELETE SET NULL,
    canal             VARCHAR(10) NOT NULL,
    estado_envio      VARCHAR(10) NOT NULL,
    fecha_envio       TIMESTAMPTZ NOT NULL DEFAULT now(),
    CHECK (canal IN ('CORREO','PUSH')),
    CHECK (estado_envio IN ('ENVIADA','FALLIDA')),
    CHECK (canal = 'PUSH' OR telefono_push_num IS NULL)
);

CREATE TABLE programacion_reporte (
    num               SERIAL      PRIMARY KEY,
    organizacion_num  INTEGER     NOT NULL REFERENCES organizacion(num),
    area_num          INTEGER     REFERENCES area(num),
    creada_por        INTEGER     NOT NULL REFERENCES usuario(num),
    frecuencia        VARCHAR(10) NOT NULL,
    destinatarios     INTEGER[]   NOT NULL,
    activa            BOOLEAN     NOT NULL DEFAULT TRUE,
    proxima_ejecucion TIMESTAMPTZ NOT NULL,
    ultima_ejecucion  TIMESTAMPTZ,
    fecha_creacion    TIMESTAMPTZ NOT NULL DEFAULT now(),
    CHECK (frecuencia IN ('DIARIA','SEMANAL','MENSUAL')),
    CHECK (cardinality(destinatarios) > 0)
);

CREATE TABLE reporte (
    num              SERIAL       PRIMARY KEY,
    organizacion_num INTEGER      NOT NULL REFERENCES organizacion(num),
    area_num         INTEGER      REFERENCES area(num),
    generado_por     INTEGER      REFERENCES usuario(num),
    origen           VARCHAR(12)  NOT NULL,
    periodo_inicio   TIMESTAMPTZ  NOT NULL,
    periodo_fin      TIMESTAMPTZ  NOT NULL,
    cifras           JSONB        NOT NULL,
    contenido        TEXT         NOT NULL,
    archivo_pdf      VARCHAR(255),
    fecha_generacion TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CHECK (origen IN ('MANUAL','PROGRAMADO','AGENTE')),
    CHECK (periodo_inicio < periodo_fin)
);

CREATE INDEX idx_dispositivo_area ON dispositivo (area_num);
CREATE INDEX idx_usuario_organizacion ON usuario (organizacion_num, area_num);
CREATE INDEX idx_programacion_activa ON programacion_reporte (activa, proxima_ejecucion);

-- ---------- Datos iniciales de los catálogos ----------
```

```
INSERT INTO rol (codigo, nombre, descripcion) VALUES
    ('ADMINPLAT', 'Administrador de plataforma', 'Equipo Núcleo: alta, baja y suspensión de
organizaciones'),
    ('ADMIN',     'Administrador', 'Áreas, dispositivos, usuarios, permisos y umbrales de su
organización'),
    ('SUPER',     'Supervisor',    'Operadores de su área, umbrales, intervalo de muestreo y alertas'),
    ('ANALISTA',  'Analista',      'Histórico de mediciones y reportes con IA'),
    ('OPER',      'Operador',      'Monitoreo en tiempo real de su dispositivo asignado');

INSERT INTO estado_usuario (codigo, nombre) VALUES
    ('ACT', 'Activo'), ('INA', 'Inactivo'), ('BLQ', 'Bloqueado');

INSERT INTO estado_dispositivo (codigo, nombre, descripcion) VALUES
    ('LINEA',    'En línea',      'Envía lecturas con normalidad'),
    ('CALENT',   'Calentando',    'Primeros 20 minutos del CCS811 tras el arranque'),
    ('SINCONEX', 'Sin conexión',  'Más de 3 intervalos de muestreo sin lecturas'),
    ('BATBAJA',  'Batería baja',  'Nivel de batería bajo'),
    ('MANT',     'Mantenimiento', 'Fuera de servicio por mantenimiento'),
    ('BAJA',     'Dado de baja',  'Ya no se utiliza');

INSERT INTO estado_sensor (codigo, nombre) VALUES
    ('OPERA', 'Operando'), ('CALIB', 'Requiere calibración'), ('FALLA', 'Falla');

INSERT INTO variable_ambiental (codigo, nombre, unidad, rango_min, rango_max) VALUES
    ('TEMP', 'Temperatura',              '°C',  -40,    80),
    ('HUM',  'Humedad relativa',         '%HR',   0,   100),
    ('AIRE', 'Calidad del aire (eCO2)',   'ppm',   0, 10000);
```

Anexo B. Script de nucleo_iot (TimescaleDB 2.31)

Crea la tabla de lecturas y la convierte en hypertable. Requiere la extensión timescaledb instalada en el servidor. La tabla y sus restricciones se probaron en PostgreSQL; la llamada a create_hypertable se ejecuta en el servicio de TimescaleDB.

```
-- ============================================================
-- Núcleo · nucleo_iot (TimescaleDB 2.31 sobre PostgreSQL 17)
-- Lecturas de los dispositivos como serie de tiempo
-- ============================================================
CREATE EXTENSION IF NOT EXISTS timescaledb;

CREATE TABLE lectura (
    num             BIGINT       GENERATED ALWAYS AS IDENTITY,
    fecha_hora      TIMESTAMPTZ  NOT NULL,              -- hora de medición
    sensor_num      INTEGER      NOT NULL,              -- ref. lógica a nucleo_db.sensor(num)
    variable_codigo VARCHAR(10)  NOT NULL,              -- ref. lógica a
nucleo_db.variable_ambiental(codigo)
    valor           NUMERIC(8,2) NOT NULL,
    recibida_en     TIMESTAMPTZ  NOT NULL DEFAULT now(),
    estabilizada    BOOLEAN      NOT NULL DEFAULT TRUE, -- FALSE durante el calentamiento del CCS811
    PRIMARY KEY (num, fecha_hora),                      -- la llave debe incluir la columna de
partición
    CHECK (variable_codigo IN ('TEMP','HUM','AIRE')),
    CHECK (  (variable_codigo = 'TEMP' AND valor BETWEEN -40 AND 80)
          OR (variable_codigo = 'HUM'  AND valor BETWEEN 0 AND 100)
          OR (variable_codigo = 'AIRE' AND valor BETWEEN 0 AND 10000))
);

-- convierte la tabla en hypertable particionada por día
SELECT create_hypertable('lectura', by_range('fecha_hora', INTERVAL '1 day'));

-- histórico, gráficas de tendencia y cifras de reportes
CREATE INDEX idx_lectura_consulta
    ON lectura (sensor_num, variable_codigo, fecha_hora DESC);
```